Verification guide
Rank-five rings through multiplicity twenty

17 October 2026

Research notice. AI-generated research · independent verification pending. The results, proofs, code and data have not yet been independently verified by human experts or formalized in Lean. Treat them as provisional; automated checks establish only their stated computational scope.

1 What this release certifies

The pinned `fusion-ring-census` repository [1] records 40999 pairwise nonisomorphic rank-five fusion rings through multiplicity twenty. The checkers validate the release tensors against the ring axioms. Exactly seventeen match the constructed unitary models. The other 40982 have verified general exclusion certificates. In particular, the two tensors at local indices 5073 and 5076 at exact multiplicity eighteen have general certificates: a new inert-prime parity theorem forces any categorification to be spherical, and their existing exact spherical-character certificates then exclude it. Thus general, pivotal, spherical, pseudo-unitary and unitary categorification is classified through twenty. Search timeouts are not exclusion certificates.

2 Environment and complete verification commands

The complete replay package checks all 40999 rings, 40982 exclusion certificates, and 17 model matches. Use Python 3.12, a C++17 compiler, and the package requirements (SymPy 1.14.0, NumPy 2.3.5 and SciPy 1.17.0). From the classification repository root run

```
python tools/restore_data.py
python evidence/prepublication_20261017/rank5/replay.py --check-package
python evidence/prepublication_20261017/rank5/replay.py --jobs 2
python -O evidence/prepublication_20261017/rank5/replay.py --jobs 2
```

The package check verifies every extracted member and its agreement with the published counts. The two replay commands check the full pinned census, all exclusion certificates and positive models, and the secondary rejection controls. Proof conditions use explicit exceptions, so optimized Python retains them. Both recorded complete runs passed and produced byte-identical ledgers. Their digest is recorded in the package metadata.

Fresh receipts and ledgers are written to `rank5-replay-output/`; use `-output-dir PATH` to select another location. The exact verifier uses rational and algebraic arithmetic; numerical optimization is used only in certificate discovery. A nonzero exit or an exception is a failed check. No conclusion is drawn from incomplete execution or a timeout. Source hashes and successful computations identify and test the encoded arguments; they do not replace independent mathematical review.

3 Census input and archival labels

The finite classification uses the pinned census repository [1]; its rank-five counts continue through multiplicity twenty. The pinned rank-five tensor and count files are bundled under `data/pinned_repository/rank5/`. The local file `data/classification_tensors.json` retains the basis choices used by the exclusion certificates. The verifier checks both the pinned file hash and an exact isomorphism-class crosswalk before checking any categorification certificate. The complete tensors and fingerprints address those certificates; there is no second census computation.

Every local row stores a full tensor `N`, multiplicity `m`, historical or appended label `j`, and local ordinal `local_index`. Existing Vercleyen–Slingerland matches retain the first historical source index and basis. At multiplicity eighteen no source rows were present, so the two index fields agree. Always select a tensor by its full multiplication coefficients if a citation or lookup crosses data releases.

The first full verification command checks the crosswalk automatically. The extension command checks the pinned repository order directly for every exact-multiplicity-nineteen and -twenty tensor and every general exclusion certificate. The files `data/decisions_all.json` and `.csv` collect the 40999 decisions; `repository_index` refers only to direct pinned-source records, whereas `j` and `local_index` in the historical segment retain their archival meanings. To rerun just this comparison using the bundled copy of the pinned repository file, run

```
python software/compare_pinned_census.py \
    --report verification/census_crosswalk.json
```

The comparison minimizes every complete tensor under all 24 unit-preserving basis permutations. The included report records equal sets for all multiplicities through eighteen, together with SHA-256 hashes of both input files. Raw labeled-tensor fingerprints are not invariant under a basis permutation.

4 The exclusion certificate contract

The dictionary `certificates/exclusions.json` is keyed by the SHA-256 fingerprint of the complete stored tensor. A fingerprint is an address only: the verifier recomputes the tensor's hypotheses and the certificate. It cannot establish nonexistence by itself. No exclusion criterion accepts `m`, `j` or a previous status as input.

The arithmetic leaves contain a factor/divisibility witness, finite-prime cyclotomic obstruction or nonintegral fusion-subring index. Positive branches verify weak integrality or the two-FP-values reduction, then central induction. Quadratic-parity certificates recover A_i from the square rows, verify all FP multiplication equations, squarefreeness, conductor congruence, the Casimir polynomial, its mod-two zero multiplicity, the square-row kernel and irreducibility. Inert-prime parity certificates add an exhaustive four-element-field character check and central divisibility filter; they recompute the exact spherical obstruction. No fixed radicand or list of labels is assumed by either criterion.

A spherical certificate exhausts the irreducible character components of the commutative rational algebra. A general signed certificate first checks that spherical certificate, then regenerates every associative signed tensor in the positive-second-indicator normalization and every remaining dimension-character component. Timeouts, omitted branches, incorrect field factors or missing low matrices cause verification to fail. The parity theorems replace this search where their hypotheses hold; they do not assert pivotality for arbitrary rank-five rings.

For low induction the checker recomputes every bounded column of required dimension and every row-sum completion. Native Gram pruning discards only a negative entry or

principal minor. High columns satisfy the full rank-one Gram inequality and the algebraic dimension tests. In a singular remainder the kernel is imposed before a nonsingular principal block is used. The joint trace box includes an all-zero type for all larger orders. This avoids falsely bounding a category's exponent by its moment height.

For every low matrix and permitted second indicator the finite-indicator certificate supplies an integer vector y. After reconstructing the complete integer system $Ax = b$, the verifier checks $A^{\mathsf{T}}y \geq 0$ and $b^{\mathsf{T}}y < 0$. Rational and quadratic fields use the same code path; all available rational indicator bounds through height six are included. The cyclic-four criterion uses the same relative traces after the proved mixed-support and central-involution reductions. Its balanced Galois branches instead check an explicitly nonintegral fusion coefficient. Every support and indicator branch is accounted for.

5 Applying the criterion to another tensor

The discovery driver does not look up the decision ledger. It either matches a constructed positive model or applies the same tensor criteria and exactly verifies the returned certificate. For example,

```
python software/discover.py --multiplicity 18 --index 5073 \
    --height 6 --seconds 120 \
    --output verification/inert_example.jsonl
```

The index in this command is the local ordinal stored in `local_index`. An arbitrary external rank-five tensor can instead be supplied as a JSON list of five 5×5 integer matrices:

```
python software/discover.py --tensor new_tensor.json \
    --height 6 --seconds 120 --output verification/new.jsonl
```

A JSON object with an `N` or `tensor` field is also accepted. For the displayed example, inert-prime parity and the spherical-character certificate apply automatically from the tensor; the historical index has no role in the criterion. A separate independent residue-field enumeration is available as `python software/audit_inert_residues.py`. For other inputs the total per-tensor allowance is separate from the signed subprocess allowance, which is half of it. An unsupported field, absent separating ray, exception or timeout produces `unresolved`, not `excluded`. Raising the budget or height is permitted; a new successful result must still pass the exact checker.

To continue beyond twenty, take the next verified tensor set from the repository census [1], construct row records by the local canonical convention, and pass that JSON through `-census`. Its mathematical criteria do not need new index-specific functions. `export_tables.py` regenerates the present release's manuscript tables and CSV ledger only after a successful full verification. A future release must explicitly update its endpoint assertions and rerun all certificates.

6 Exactness, controls and scope of trust

All proof arithmetic is integral or rational, including factorization and real-root interval comparisons. NumPy is used for bounded integer tensor operations, not eigenvalue evidence: with rank five and bound thirty-two each multiplication sum is at most $5 \cdot 32^2$. The signed generator computes a bound for each polynomial before using 64-bit evaluations; diagnostic counters fail on overflow. Native low-Gram inputs are bounded by 10^6; their four-dimensional determinant sums fit in checked 128-bit ranges. High-Gram bounds are

validated by the Python wrapper before native execution. An overflow or failed divisibility halts the computation.

The complete runs additionally compare native Gram-vector enumeration with arbitrary-integer Cartesian enumeration on four matrices, compare native low joins with unpruned Python joins and exact minors on four instances, and deliberately corrupt new criterion certificates. The known positive $K(C_4, 4)$ model has an exactly checked rational feasible branch in the cyclic central-support system. This control invokes no numerical linear programming. Scalar tests check the Farkas orientation and reject floating-point rays. These are supplementary controls, not claims of formal verification of the compiler, algebra library or categorical proofs.

The code does not construct every unitary associator. Positive existence is established by the cited categorical constructions; the code verifies their fusion tensors and exact census matches. Likewise, a certificate checker verifies applications of proved necessary conditions, not the proofs of those conditions in a theorem prover. The mathematical proof, software and data are separate, inspectable parts of the result.

7 Files and rebuilding

`README.md` gives the directory map and principal commands. `data/decisions_all.json` and its CSV companion record all five existence statuses through twenty; `data/decisions.json` preserves the historical tensor labels through eighteen. `data/extension19_20/full.jsonl` contains the directly indexed repository tensors and exact certificates. `data/frontier.json` preserves the two historically difficult tensors, formal codegrees, FP dimensions, spherical certificates and discovery diagnostics. Their complete general certificates are in one tensor-indexed file. All source inputs, run measurements and version information are recorded in `provenance/`. Obsolete case-specific scripts and status tables are retired, not left as contradictory alternative entry points.

The LaTeX sources and PDFs for the manuscript and this guide are in `papers/`. Run `python build_pdfs.py` from the root to compile them. For current counts use the pinned repository [1].

AI use disclosure

The research, code, and manuscript were generated with AI assistance. Independent human verification and Lean formalization are pending.

References

[1] *Fusion-ring census*, repository, commit `b86eb6dc6eeeafe5fc5f6906d7b074e0322e9726`, `results/rank5/counts.csv` and `results/rank5/tables.txt.gz` (2026).